

Postgres Basic Queries & Clauses

CRUD + Insert Operations:

Create Table:

-- Create a table (if it doesn't already exist)

```
CREATE TABLE IF NOT EXISTS employees (
```

```
  id SERIAL PRIMARY KEY,
```

```
  name VARCHAR(100) NOT NULL,
```

```
  department VARCHAR(50),
```

```
  salary NUMERIC(10, 2)
```

```
);
```

- Remember we cannot violate the primary key
- Remember we cannot have more than one table with same name
- Remember we cannot leave “Not Null” column empty in any record

Read Data:

-- Select all records from the table

```
SELECT * FROM employees;
```

-- Select specific columns

```
SELECT id, name, salary FROM employees;
```

-- Filter records using WHERE clause

```
SELECT * FROM employees WHERE department = 'Engineering';
```

-- Sort records using ORDER BY

```
SELECT * FROM employees ORDER BY salary DESC;
```

-- Limit the number of records returned

```
SELECT * FROM employees LIMIT 5;
```

Update Data:

-- Update a specific record

```
UPDATE employees
```

```
SET salary = 80000.00
```

```
WHERE id = 1;
```

-- Update multiple columns

```
UPDATE employees
```

```
SET department = 'HR', salary = 85000.00
```

```
WHERE name = 'John Doe';
```

-- Update all records (use with caution!)

```
UPDATE employees
```

```
SET salary = salary * 1.10; -- Give a 10% raise to all employees
```

Delete Data:

-- Delete a specific record

```
DELETE FROM employees
```

```
WHERE id = 1;
```

-- Delete records based on a condition

```
DELETE FROM employees
```

```
WHERE department = 'HR';
```

-- Delete all records (use with caution!)

DELETE FROM employees;

Insert Data:

-- Insert a new record

```
INSERT INTO employees (name, department, salary)
VALUES ('John Doe', 'Engineering', 75000.00);
```

-- Insert multiple new records

```
INSERT INTO employees (name, department, salary)
VALUES
    ('John Doe', 'Engineering', 75000.00),
    ('Jane Smith', 'Marketing', 65000.00),
    ('Alice Johnson', 'HR', 70000.00);
```

Some important clauses & their purpose

Clause	Purpose
GROUP BY	Groups rows for aggregation.
HAVING	Filters groups after aggregation.
ORDER BY	Sorts the result set.
WHERE	Filters rows before aggregation.
DISTINCT	Removes duplicate rows.
LIMIT/OFFSET	Limits the number of rows returned.
LIKE	Filters rows based on pattern matching.

Clause	Purpose
IS NULL	Filters rows based on NULL values.